Chains & LCEL

How to wire LLM steps together with the pipe operator, run them in parallel, pass data between them, and give a chatbot a memory — every example shown for both Ollama and the Gemini API.

THIS VOLUME COVERS

Phase 2 — Chains & LCEL
Project: a multi-turn assistant

Where we are

In Part 1 you called a model, learned `invoke` versus `stream`, and met the `PromptTemplate`. You even built your first tiny pipeline with the `|` operator. This part is about that operator in full.

LCEL — the LangChain Expression Language — is the modern, declarative way to connect steps. If you have used a Unix shell, the mental model is exactly `cat file | grep word | sort`: each stage's output becomes the next stage's input. In LangChain that reads `prompt | model | parser`. By the end of this part you will compose multi-step chains, run independent steps in parallel, inject your own Python functions into a chain, take a first look at **document loaders**, and give a chatbot **memory** so it remembers the conversation.

> **A note on what changed at 1.0**
>
> Older tutorials build chatbots with `LLMChain`, `ConversationChain`, and `ConversationBufferMemory`. Those were deprecated and moved to the separate `langchain-classic` package at the 1.0 release. Everything in this part uses the current LCEL and message-history interfaces — if you see those old class names online, you are reading pre-1.0 material.

THE ENGINE TOGGLE, ONE MORE TIME

As before, "choose your engine" means picking one of these two lines. The rest of every example is identical.

PYTHON

```python
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model

load_dotenv()

llm = init_chat_model("ollama:llama3.1", temperature=0)           # local (Ollama)
# llm = init_chat_model("google_genai:gemini-2.5-flash", temperature=0)  # cloud (Gemini)
```

PHASE 2

Chains & LCEL

Real applications are rarely a single model call. They format an input, call a model, parse the result, maybe call another model, and combine everything. LCEL is how you express that flow cleanly — and because every piece speaks the same interface, the whole chain gets `invoke`, `stream`, and `batch` for free.

2.1 Everything is a Runnable

The foundation of LCEL is one idea: almost every LangChain object — a prompt template, a chat model, an output parser — implements the same protocol called **Runnable**. A Runnable is anything you can `.invoke()`. Because they all share that protocol, the `|` operator can glue any Runnable to the next: it feeds the left side's output into the right side's input, producing a new, larger Runnable.

PYTHON

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Give one fun fact about {animal}.")

chain = prompt | llm | StrOutputParser()   # three Runnables joined into one

print(chain.invoke({"animal": "octopus"}))
```

The crucial payoff: the combined `chain` is itself a Runnable, so it has the very methods you learned in Part 1. Nothing new to memorise.

PYTHON

```python
# Stream the chain's output token-by-token
for chunk in chain.stream({"animal": "octopus"}):
    print(chunk, end="", flush=True)
print()

# Run many inputs through the same chain
print(chain.batch([{"animal": "cat"}, {"animal": "owl"}, {"animal": "bee"}]))
```

Why LCEL over hand-written code

You could call the prompt, the model, and the parser yourself with three lines of plain Python. Composing them as a chain buys you uniform streaming, batching, async, retries, and (in Part 5) automatic tracing — without writing any of that plumbing. The chain is the unit LangChain knows how to optimise and observe.

2.2 Output parsers: turning replies into usable data

A chat model returns an `AIMessage`. An **output parser** is the final stage that converts that message into the shape your program actually needs. You already met the simplest one, `StrOutputParser`, which extracts the plain text. The next most useful turns the model's reply into a Python dictionary.

PYTHON

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import JsonOutputParser

prompt = ChatPromptTemplate.from_template(
    'For "{place}", return ONLY JSON with keys "city" and "country".'
)

chain = prompt | llm | JsonOutputParser()

data = chain.invoke({"place": "the Eiffel Tower"})
print(data)              # {'city': 'Paris', 'country': 'France'}
print(data["country"])   # France  --  a real dict you can index
```

This is identical for both engines. The parser sits at the end of the pipe, so the rest of the chain never changes when you decide you want structured data instead of a string.

Parsers are just Runnables too

That is why they slot into the pipe. LangChain ships several (string, JSON, lists, and typed/validated objects). When you need strict, schema-checked output, the model method `llm.with_structured_output(Schema)` is the robust modern option — we use it in the RAG and agent parts where structure matters most.

2.3 Branching, parallelism, and your own functions

A straight pipe is a single production line. Three small helpers from `langchain_core.runnables` let you build richer shapes: run steps side by side, pass the original input forward, and drop ordinary Python functions into the flow.

RunnableParallel — do several things at once

Wrap a dict of chains and they all execute against the same input simultaneously; the result is a dict with the same keys. This is faster than running them one after another when the steps are independent.

PYTHON

```python
from langchain_core.runnables import RunnableParallel
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

summary = ChatPromptTemplate.from_template("Summarize {topic} in one sentence.") | llm |
StrOutputParser()
quiz    = ChatPromptTemplate.from_template("Write one quiz question about {topic}.") | llm |
StrOutputParser()

both = RunnableParallel(summary=summary, quiz=quiz)   # run together

result = both.invoke({"topic": "photosynthesis"})
print(result["summary"])
print(result["quiz"])
```

A plain dict becomes a RunnableParallel

LCEL auto-converts a dict of Runnables into a `RunnableParallel`. So `{"summary": summary, "quiz": quiz} | next_step` works without naming the class. You will see this shorthand everywhere — including the RAG chain in Part 3.

RunnableLambda — insert any Python function

Wrap a normal function so it becomes a chain stage. The function takes the previous step's output and returns the next step's input.

PYTHON

```python
from langchain_core.runnables import RunnableLambda

def shout(text: str) -> str:
    return text.upper() + "!"

# Clean up the model's text before it leaves the chain
chain = prompt | llm | StrOutputParser() | RunnableLambda(shout)
print(chain.invoke({"animal": "fox"}))
```

RunnablePassthrough — carry the input along

Sometimes a later step needs the *original* input, not just the previous step's output. `RunnablePassthrough` forwards the input unchanged, and `.assign()` adds new keys while keeping the existing ones.

PYTHON

```python
from langchain_core.runnables import RunnablePassthrough

# Keep "text", and add a computed "length" key alongside it
add_length = RunnablePassthrough.assign(length=lambda x: len(x["text"]))

print(add_length.invoke({"text": "hello"}))   # {'text': 'hello', 'length': 5}
```

This trio — parallel, lambda, passthrough — is the toolkit for the "retrieve context, then answer" pattern at the heart of RAG, which is exactly where Part 3 begins.

2.4 Document loaders (a first look)

Chains become powerful once they work over *your* data. A **document loader** reads a source — a text file, a PDF, a web page — and returns a list of `Document` objects. Each `Document` has two parts: `page_content` (the text) and `metadata` (where it came from). Loaders live in the `langchain-community` package.

TERMINAL

```
pip install -U langchain-community pypdf
```

PYTHON

```python
from langchain_community.document_loaders import TextLoader, PyPDFLoader

# A plain text file -> usually one Document
docs = TextLoader("notes.txt").load()
print(docs[0].page_content[:80])
print(docs[0].metadata)          # e.g. {'source': 'notes.txt'}

# A PDF -> one Document per page
pages = PyPDFLoader("report.pdf").load()
print(len(pages), "pages loaded")
```

This is the on-ramp to RAG

Loading is step one of “chat with a PDF.” In Part 3 we take these `Document` objects, split them into chunks, turn the chunks into embeddings (with **Ollama** or **Gemini** embedding models), store them in a vector database, and retrieve the most relevant pieces to answer questions. For now, simply note that loaders hand you clean `Document` objects.

2.5 Memory: making a chatbot remember

Here is a surprise that trips up every beginner: **LLMs are stateless**. The model keeps no record of your last message. The only reason a chatbot "remembers" is that the application re-sends the whole conversation on every turn. "Memory" is just the machinery that stores past messages and feeds them back in. There are two clean ways to do this in current LangChain.

Approach A — keep the messages in a list (the concept, bare)

The clearest way to see how memory works is to manage the list yourself: append each new message, and pass the entire list to the model every time.

PYTHON

```python
from langchain_core.messages import SystemMessage, HumanMessage

history = [SystemMessage("You are a friendly assistant named Iris.")]

while True:
    text = input("you > ").strip()
    if text.lower() in {"quit", "exit"}:
        break
    history.append(HumanMessage(text))     # remember what the user said
    reply = llm.invoke(history)            # send the WHOLE history each turn
    print("Iris >", reply.content)
    history.append(reply)                  # remember the AI's answer too
```

Because the model now receives every prior message, it can answer "what is my name?" correctly. The cost is that the message list keeps growing — long conversations eventually need trimming or summarising.

Approach B — let LCEL manage history per session

For anything reusable, LangChain provides `RunnableWithMessageHistory`. It wraps a chain and automatically reads and writes the history for a given `session_id`, injecting past messages into a `MessagesPlaceholder` in your prompt. This is the idiomatic LCEL pattern and it scales to many users, each with their own session.

PYTHON

```python
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.output_parsers import StrOutputParser
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a friendly assistant named Iris."),
    MessagesPlaceholder("history"),     # past turns are injected here
    ("human", "{input}"),
])
chain = prompt | llm | StrOutputParser()

store = {}                              # session_id -> history object
def get_history(session_id: str) -> InMemoryChatMessageHistory:
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

chatbot = RunnableWithMessageHistory(
    chain, get_history,
    input_messages_key="input",
    history_messages_key="history",
)

cfg = {"configurable": {"session_id": "user-1"}}
print(chatbot.invoke({"input": "My name is Sam."}, config=cfg))
print(chatbot.invoke({"input": "What is my name?"}, config=cfg))  # -> Sam
```

What about LangGraph?

For production — durable memory that survives restarts, persists to a database, and supports trimming/summarising — the recommended path is LangGraph's persistence (a "checkpointer" such as `MemorySaver`). LangChain's agents are built on it. We meet LangGraph properly in Part 4; the two approaches above are perfect for learning and for single-process apps.

PHASE 2 PROJECT

Build a multi-turn assistant

Now we combine the whole phase into one program: a command-line assistant that holds a real conversation. It uses a `ChatPromptTemplate` with a `MessagesPlaceholder`, pipes it into your chosen model and a `StrOutputParser`, wraps the chain in `RunnableWithMessageHistory` so it remembers each turn, and `stream`s the reply for a live feel.

PYTHON (MULTI_TURN_ASSISTANT.PY)

```python
# multi_turn_assistant.py
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.output_parsers import StrOutputParser
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

load_dotenv()

# --- choose your engine ------------------------------------------------
llm = init_chat_model("ollama:llama3.1", temperature=0.3)
# llm = init_chat_model("google_genai:gemini-2.5-flash", temperature=0.3)
# -----------------------------------------------------------------------

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are Iris, a friendly and concise assistant. "
               "Use the conversation so far to stay consistent."),
    MessagesPlaceholder("history"),
    ("human", "{input}"),
])

chain = prompt | llm | StrOutputParser()

store = {}
def get_history(session_id: str) -> InMemoryChatMessageHistory:
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

assistant = RunnableWithMessageHistory(
    chain, get_history,
    input_messages_key="input",
    history_messages_key="history",
)

config = {"configurable": {"session_id": "user-1"}}

print("Multi-turn assistant (Iris). Type 'quit' to exit.\n")
while True:
    text = input("you > ").strip()
    if text.lower() in {"quit", "exit"}:
        print("Iris > Goodbye!")
        break
    if not text:
        continue
    print("Iris > ", end="", flush=True)
    for chunk in assistant.stream({"input": text}, config=config):
        print(chunk, end="", flush=True)
    print("\n")
```

Run it and watch the memory work across turns:

TERMINAL

```
python multi_turn_assistant.py
```

OUTPUT

```
Multi-turn assistant (Iris). Type 'quit' to exit.

you > Hi, I'm Sam and I love astronomy.
Iris > Nice to meet you, Sam! Astronomy is a wonderful field. What would
you like to talk about - planets, stars, or something else?

you > What's my name and what do I like?
Iris > You're Sam, and you love astronomy.

you > quit
Iris > Goodbye!
```

Extend it

(1) Flip the engine line and confirm Iris behaves the same. (2) Give two different `session_id` values and watch the two conversations stay separate. (3) Add a second parallel step with `RunnableParallel` that also returns a one-word mood label for each reply. Each change touches exactly one Phase 2 idea.

Part 2 recap & what comes next

You can now compose, branch, and add memory — the skills every larger LangChain app is built from.

You can now

- Explain why everything is a `Runnable` and how `|` composes them.
- Parse output into strings and JSON dicts.
- Run steps in parallel and inject your own functions with `RunnableLambda`.
- Load documents into `Document` objects.
- Give a chatbot memory two ways, and you know the production path.

Key objects introduced

- `StrOutputParser`, `JsonOutputParser`
- `RunnableParallel`, `RunnableLambda`
- `RunnablePassthrough` & `.assign()`
- `TextLoader`, `PyPDFLoader`
- `MessagesPlaceholder`, `RunnableWithMessageHistory`

Coming in Part 3 — RAG & Embeddings

This is where LangChain becomes genuinely powerful for real work. We load documents, split them into chunks, turn chunks into **embeddings** (using Ollama's `nomic-embed-text` and Gemini's embedding model), store them in a **FAISS** vector database, and retrieve the most relevant pieces to answer questions. The project is the classic **"chat with a PDF"** app — it ties the whole pipeline together.

End of Part 2 · The LangChain Practitioner's Guide